

CS & IT ENGINEERING



THEORY OF COMPUTATION

Turing Machine

Lecture No.- 01



By- Venkat sir



Recap of Previous Lecture



Topic

??

→ Contextfree languages Detection

→ closure properties of CFL

CFL $\cap$ Regular

always

CFL may (or) may not
Regular

DCFL $\cap$ Regular

→ always DCFL
may (or) may not Regular



Topics to be Covered



Topic

Turing Machine

Topic

?? Recursive language

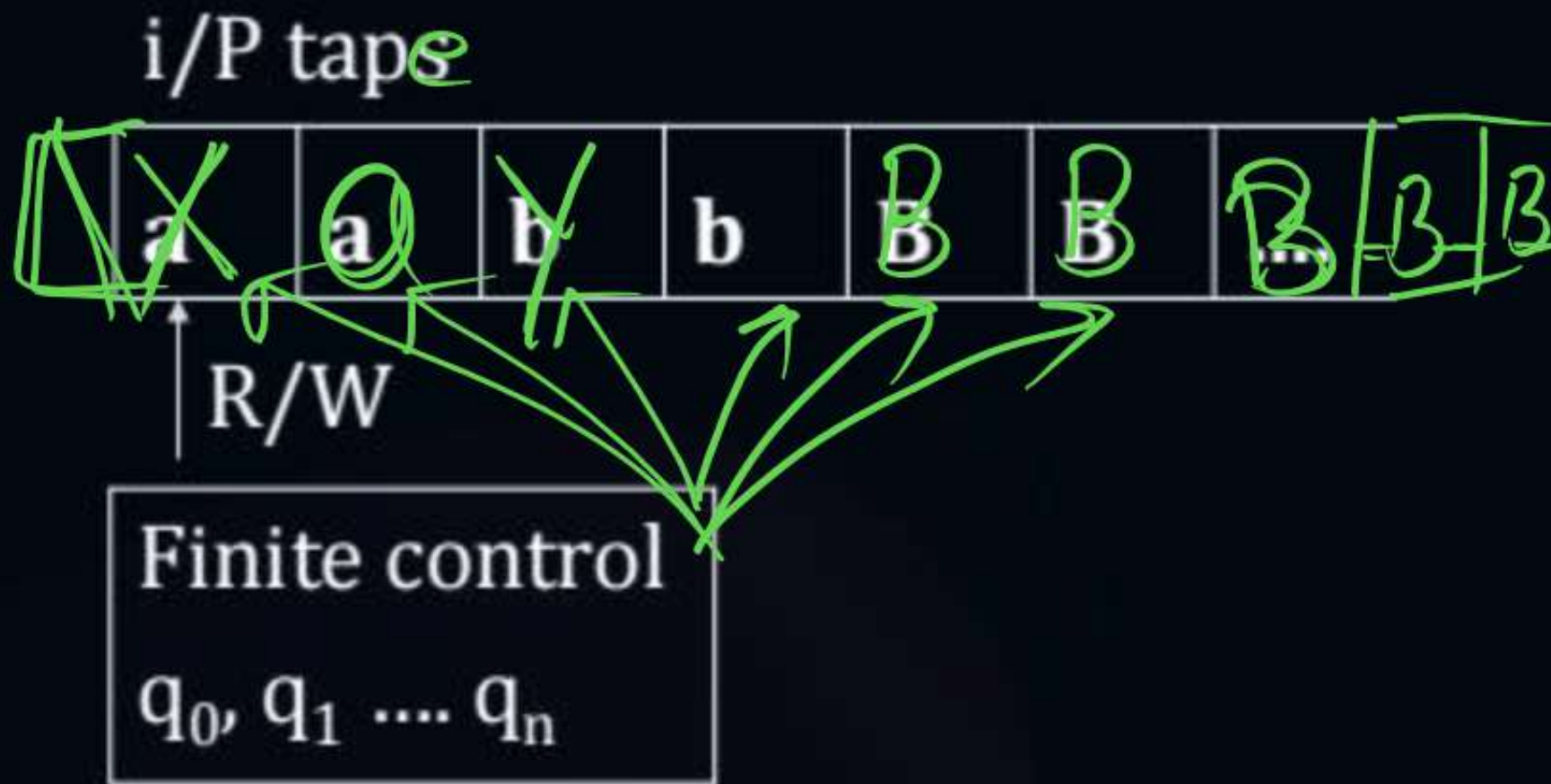
Topic

?? Recursive Enumerable language

Undecidability



Topic : Turing Machine



$\Sigma = \{a, b\}$

$\Gamma = \text{type Alphabet}$

$\{x, y, a, b, B\}$ $L = \{a^n b^n c^n d^n\}$

F.A. x

PDFA x

T.M. ✓

1. Infinite length tape
2. Turn around capability
3. Read/write capability



Topic : Turing Machine

- ① Turing machine is a mathematical model that represents general purpose computer.
- ② The problem, not solved by Turing machine ^{are} ~~or~~ not soluble by computer also.
- ③ Hence Turing machine are used to study power of a compiler.

NOTE:

Computer to finite automata, PDA, Turing having additional property they are

1. **Infinite Length tape:** Turing machine is one side closed and one side infinite.
2. **Turnaround capability:** Turing machine to turn left as well as right side.
3. **Read-Write capability:** Turing machine can replace reading symbol by other (or) same symbol.

Topic : Turing Machine

Turing Machine = $(Q, \Sigma, q_0, F, B, \Gamma, S)$

✓ Q : Finite number of state

✓ Σ : I/p alphabet

✓ q_0 : Initial state

✓ F : Set of final states

✓ B : Blank symbol

✓ Γ : Tape alphabet

S : Transition function.

Q	$\times$	Γ	$\rightarrow$	Q	$\times$	Γ	$\times \{L, R\}$
-----	----------	----------	---------------	-----	----------	----------	-------------------

$$Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

$$(q_1, a) = (q_2, x, L)$$

$$(q_2, b) = (q_3, y, R)$$



Topic : Turing Machine

$$|Q| \times |\tau| \rightarrow |Q| \times |\tau| \times \{L, R\}$$

Notaulus :

$\Rightarrow$ Transition diagram

$\Rightarrow$ Transition Table

Type of TM

(ii)

Language Recognizer

yes

no

i/p

O/P generator

$\Rightarrow$ OP

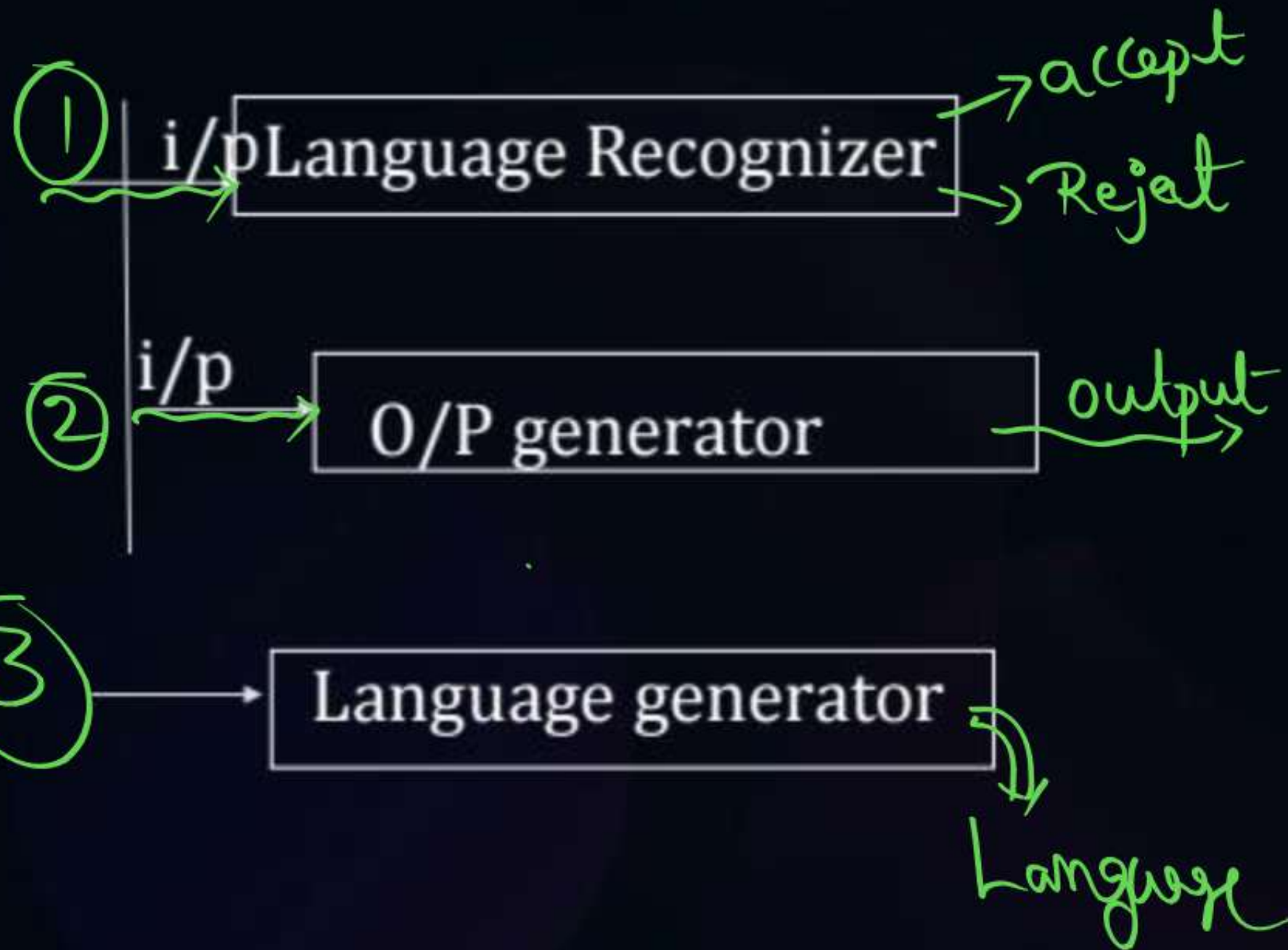
Language Generator

$\Rightarrow$ Lans



Topic : Turing Machine

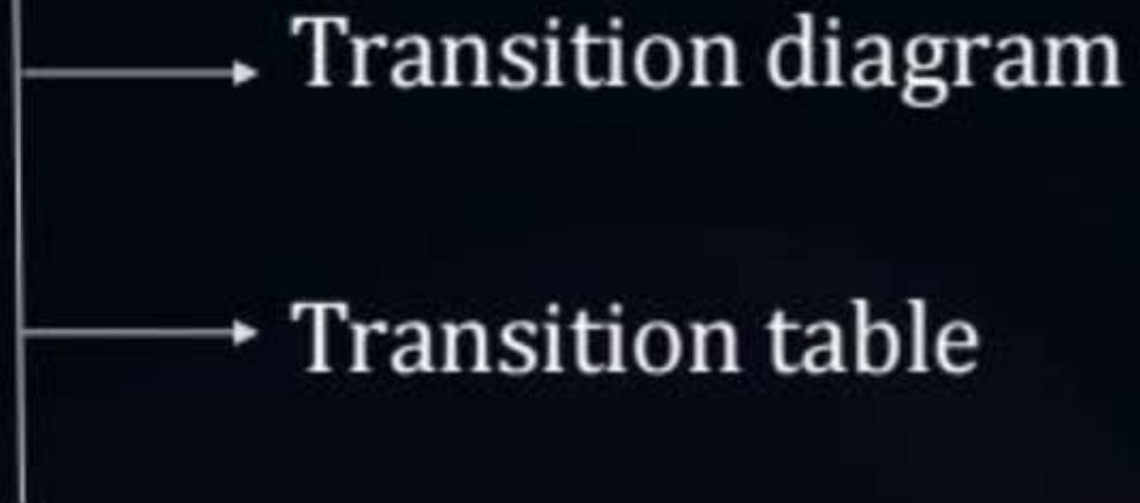
Type of Turing Machine



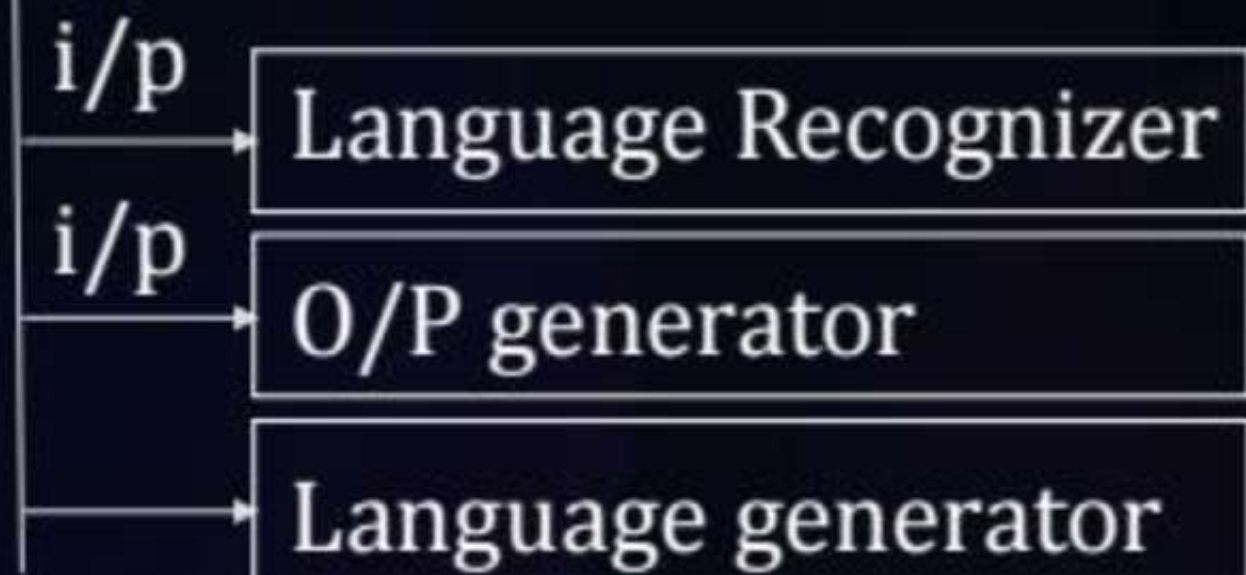


Topic : Turing Machine

Notations



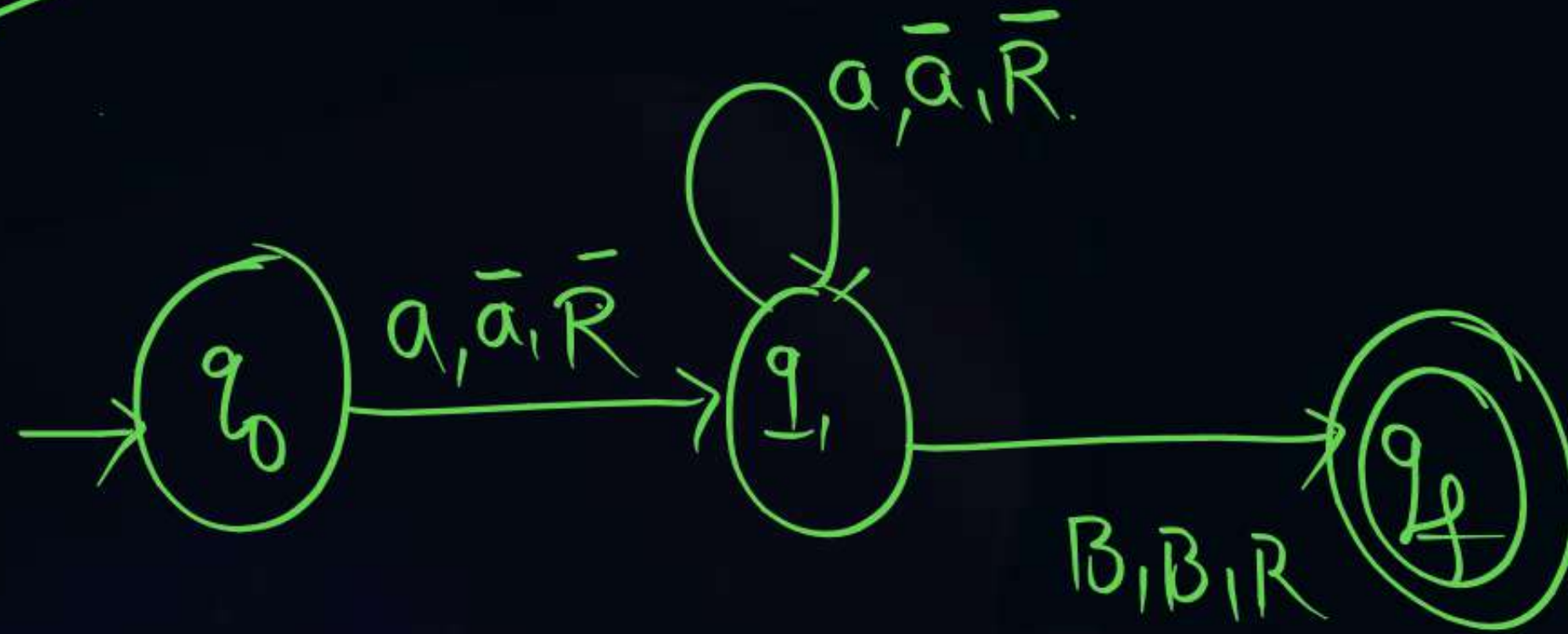
Type of Turing Machine



(Q) Construct Turing Machine for $L = \{a^n \mid n \geq 1\}$

$$Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

transition diagram

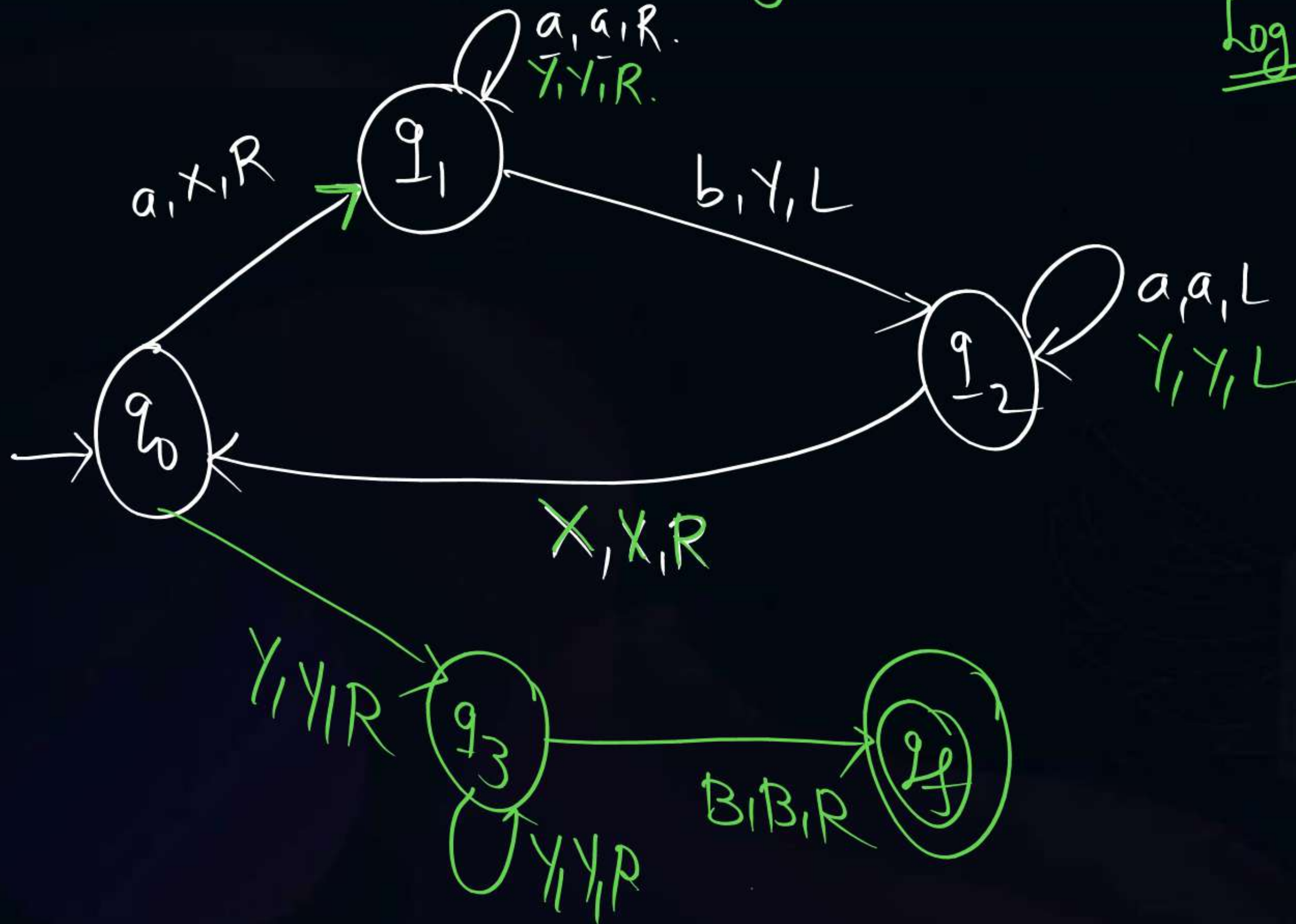


$\begin{array}{c} \boxed{a \mid a \mid a \mid a \mid B \mid B} \\ \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \quad \uparrow \\ q_0 \quad q_1 \quad q_1 \quad q_1 \quad q_2 \quad q_f \end{array}$

	✓	
q_0	(q_1, a, R)	
q_1	(q_1, a, R)	(q_f, B, R)
q_f	final state	

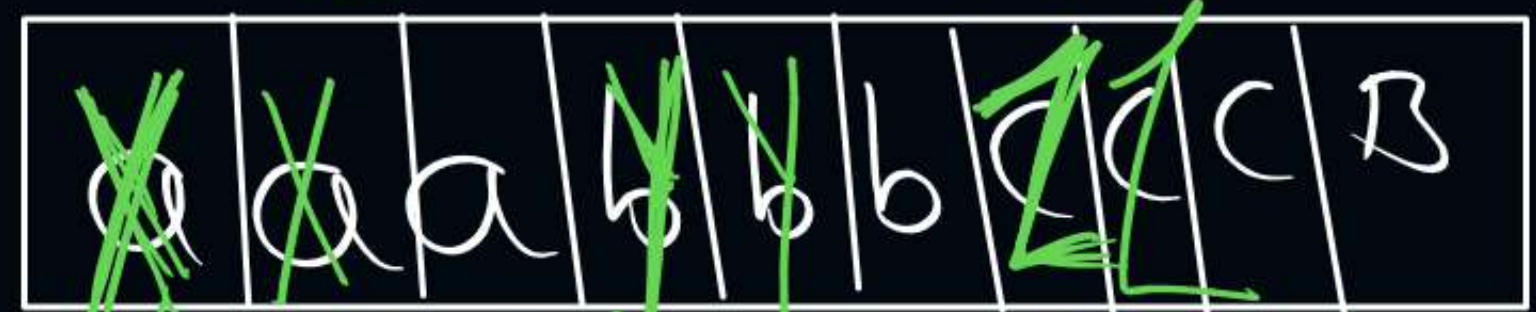
transition table

#(Q) Construct Turing Machine for $L = \{a^n b^n \mid n \geq 1\}$



(Q) Construct Turing Machine for

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

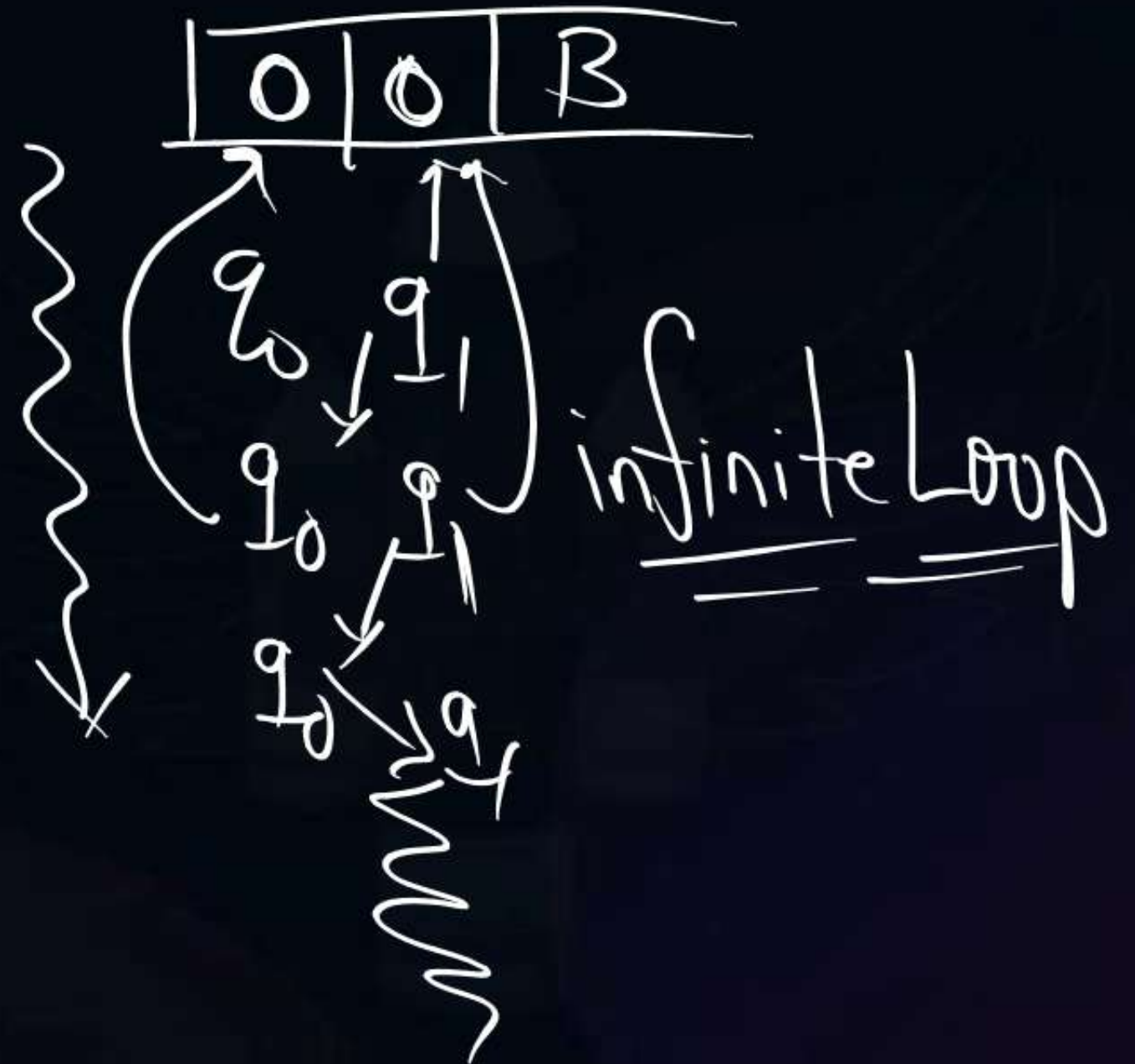


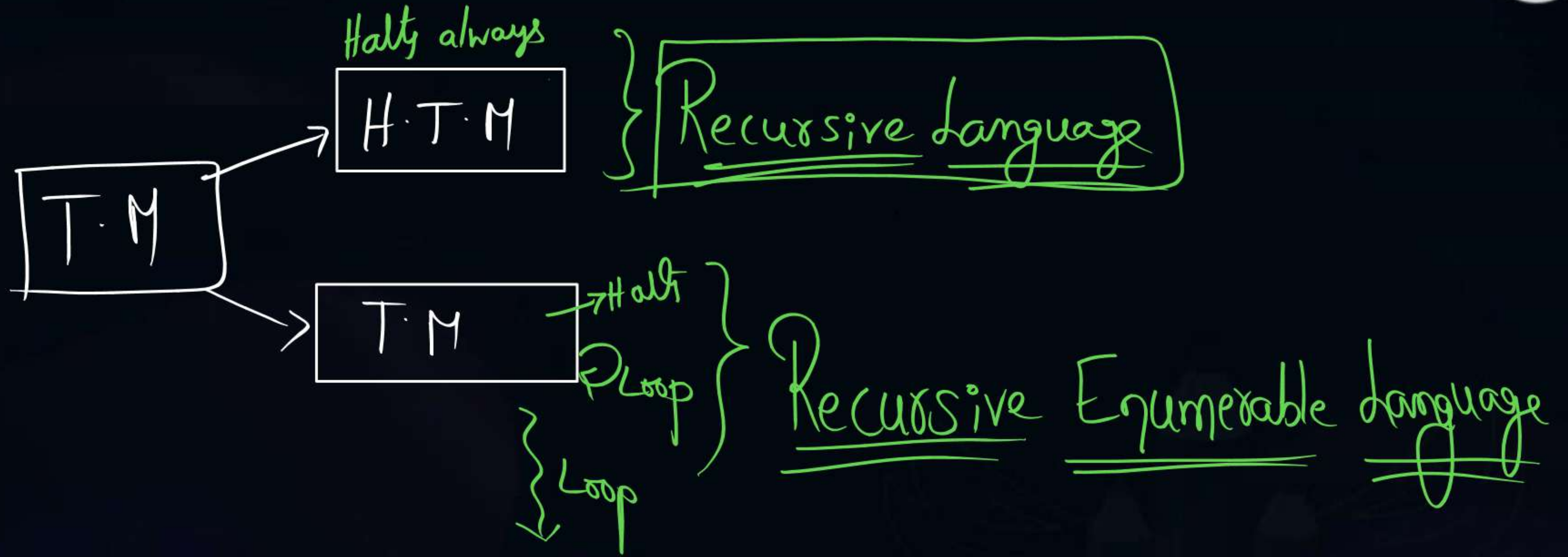
Home Work



- | | |
|---|---|
| 0 | B |
|---|---|

✓
- q_0 q_1







Topic : Recursive and Recursive Enumerable Language in TOC

Recursive Enumerable (RE) or Type-0 Language

RE languages or type-0 languages are generated by type-0 grammars.

An RE language can be accepted or recognized by Turing machine which means it will enter into final state for the strings of language and may or may not enter into rejecting state for the strings which are not part of the language.

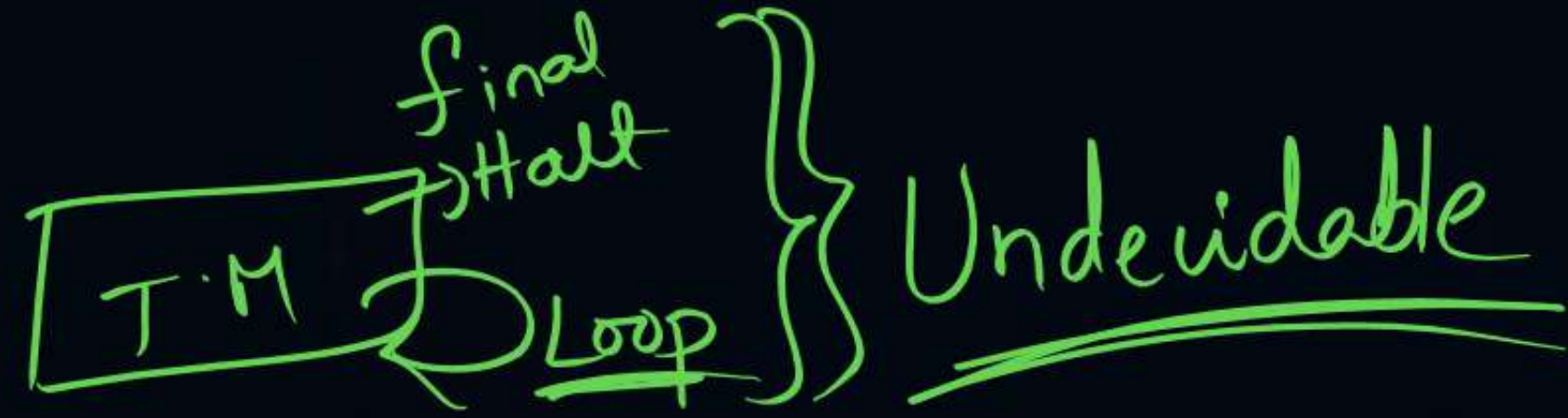
It means TM can loop forever for the strings which are not a part of the language. RE languages are also called as Turing recognizable languages.

• Recursive Language (REC)

- A recursive language (subset of RE) can be decided by Turing machine which means it will enter into final state for the strings of language and rejecting state for the strings which are not part of the Language.
- e.g.; $L = \{a^n b^n c^n \mid n \geq 1\}$
- is recursive because we can construct a turing machine which will move to final state if the string is of the form $a^n b^n c^n$ else move to non-final state.
- So the TM will always halt in this case. REC Languages are also called as Turing decidable languages.



Topic : Turing Machine



R.E.L

A language 'L' is said to be REL if there exist a Turing machine for that language, that Turing machine may halt on same i/p (or) may not halt on same i/p

→ I.e if the string is valid string of the languages then Turing Machine halts in final state and it says string is accepted.

→ If the string is not belongs to the language in the enter into infinite loop (or) halt in non final state

→ REL are called as Turing recognizable language

→ If any languages REL then it is undecidable
(number halting Turing machine exits)



Topic : Recursive Language



A lang: L is set to be recursive if there existed ^{or} Turing M/C for that always halts for on all I/P strings.

i.e. if the string is valid string of a lang that the T.H. Halts in a final state and says string is accepted.

- If the string is not belong to the lang T.M. Halts in Non- final state and it says string is rejected.
- For any lang halting T.M. exists then it is “decidable”.
- Halting TM is exactly to Algorithm.
- Hence, recursive lang also known as Turing decidable language.



Topic : CONTEXTFREE LANGUAGE

closure Properties of DCFL and CFL

<u>Operation</u>	<u>CFL</u>	<u>DCFL</u>
① Union op	✓	✗
② Concatenation op	✓	✗
③ Complement op	✗	✓ ^{DCFL}
④ Intersection op	✗	✗
⑤ Intersection with Regular ^{CFL ∩ Reg}	✓ ^{CFL}	✓ ^{DCFL}
⑥ Union with Regular	✓	✓

Operation	<u>CFL</u>	<u>DCFL</u>
⑦ Kleene closure	✓	×
⑧ Positive closure	✓	×
⑨ Difference op $[L_1 - L_2 = L_1 \cap L_2^c]$	×	×
⑩ Subset op	×	×
⑪ L -reg $[L \cap \text{reg}]$ CFL-reg	✓	✓
⑫ regular-L $[\text{reg} \cap L]$ CFL	×	✓

<u>Operation</u>	<u>CFL</u>	<u>DCFL</u>
⑬ <u>Substitution op</u>	✓	✗
⑭ <u>Homomorphism op</u>	✓	✗
⑮ <u>Inverse Homomorphism op</u>	✓	✓

#(Q) Let $P \Rightarrow \text{CFL}$
 $Q \Rightarrow \text{Regular}$ } Which of the following is always Regular?



(a) $\boxed{P \wedge Q} \Rightarrow \text{CFL}$

(b) $\boxed{P - Q} \Rightarrow P \wedge Q^c \Rightarrow \text{CFL} \wedge \text{Reg}^c \Rightarrow \text{CFL}$

(c) $\boxed{\Sigma^* - P} \times$

(d) $\boxed{\Sigma^* - Q} \Rightarrow \text{Reg}^c \Rightarrow \underline{\text{regular}}$



(Q) Let $L_1 = \{a^n b^n c a^m b^m / n, m \geq 0\}$ $L_2 = \{a^i b^j c^k / i, j, k \geq 1\}$ } $L = L_1 \cap L_2$
 then L is

(CFL & reg)

✓ (a) Regular (b)

✓ ~~(b)~~ CFL

✓ (c) non CFL

✓ (d) finite language

N.F $\Rightarrow$ Apply Condition on
 $\left\{ \begin{array}{l} \text{R.H.S of Grammar} \end{array} \right\}$

Normal form of CFG

Chomsky N.F

Greibach N.F

$\left\{ \begin{array}{l} A \rightarrow \underline{B} \underline{C} \\ A \rightarrow a \end{array} \right\}$

$a \in T$

$A \rightarrow \underline{a} \underline{V}^*$

Ex $A \rightarrow aBCDE$
 $\boxed{A \rightarrow a}$

GNF?

$$\left\{ \begin{array}{l} S \rightarrow \underline{a} \underline{AB} / \underline{aB} \\ A \rightarrow \underline{aB} \\ B \rightarrow b \end{array} \right.$$

Ex

$$S \rightarrow \underline{a} \underline{S} \underline{b} \mid \underline{a} \underline{b}$$

$(a^n b^n)$

Convert Given Grammar into
CNF Grammar. $(2 \times 4 - 1)$

$$\begin{aligned} \textcircled{1} \quad & S \rightarrow A \underline{S} \underline{B} \mid \underline{A} \underline{B} \\ & A \rightarrow a \checkmark \\ & B \rightarrow b \checkmark \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad & S \rightarrow AX \mid AB \\ & X \rightarrow SB \\ & A \rightarrow a \\ & B \rightarrow b \end{aligned}$$

$(2n-1) = 3$
 $(2 \times 2 - 1) = 3$

$\underline{ab}$ $\textcircled{4}$

$\textcircled{1} \downarrow$
 $S \rightarrow AB$

$\textcircled{2} \downarrow$
 aB

$\textcircled{3} \downarrow$
 ab

$\textcircled{4} \downarrow$
 aAB

$\textcircled{5} \downarrow$
 $aABB$

$\textcircled{6} \downarrow$
 $aabb$

$\textcircled{7} \downarrow$
 $aaab$

$\textcircled{1} \downarrow$
 $S \rightarrow AX$

$\textcircled{2} \downarrow$
 aX

$\textcircled{3} \downarrow$
 aSB

$\textcircled{4} \downarrow$
 $aABB$

$\textcircled{5} \downarrow$
 $aaBB$

$\textcircled{6} \downarrow$
 $aabb$

$\textcircled{7} \downarrow$
 $aaab$

NOTE



- ① To generate n length string from CNF grammar total derivation steps required is $(2n-1)$
[total productions required is $(2n-1)$]

NOTE

To generate n length string from GNF grammar
total number of productions required is n only
[total number of derivation steps required is n only]



2 mins Summary



Topic

Turing Machine

Topic

Topic

Topic

Topic



THANK - YOU